TECHNICAL REFERENCE & CURRICULUM GUIDE

Module 10 (Part 1): Fine-Tuning, PEFT, Alignment & Distributed Training

Module 10 Part 1 covers the engineering and mathematical principles behind customizing Large Language Models: architectural decision frameworks, full fine-tuning compute dynamics, dataset formats and synthetic curation, the Parameter-Efficient Fine-Tuning (PEFT) landscape, and the mechanics of Low-Rank Adaptation (LoRA).

10.1 Fine-Tuning vs. Prompt Engineering vs. RAG: Enterprise Decision Framework

Adapting a foundational LLM to enterprise workloads centers on three paradigms: **In-Context Prompt Engineering**, **Retrieval-Augmented Generation (RAG)**, and **Parametric Fine-Tuning (SFT/PEFT)**. Each intervenes at a different layer of the inference and optimization hierarchy.

```
                    ENTERPRISE ADAPTATION SPECTRUM
┌──────────────────────┬──────────────────────┬──────────────────────┐
│   PROMPT ENGINEERING │          RAG         │      FINE-TUNING     │
├──────────────────────┼──────────────────────┼──────────────────────┤
│ Context-Window Level │ Dynamic Knowledge Bridge │ Model Weight Mutation │
│                      │                      │                      │
│ • Zero parameter update │ • Dynamic external data │ • Mutates model weights │
│ • Zero training compute │ • Real-time DB lookup  │ • Changes tone, syntax, │
│ • High prompt token cost│ • Provenance & citation │   and domain behavior  │
│ • Volatile & transient  │ • Medium token overhead │ • Low prompt token cost │
└──────────────────────┴──────────────────────┴──────────────────────┘
```

1. The Core Trade-Off Axis: Knowledge vs. Form

- **Knowledge Retrieval (Dynamic Factual Grounding):** RAG is strictly superior when the application requires real-time information, private relational records, strict auditability, document citations, or data subject to frequent mutations (e.g., product inventory, customer CRM records). Fine-tuning an LLM to "memorize" factual tables causes catastrophic hallucinations, as neural weights store knowledge lossily through distributed associations rather than exact lookups.
- **Form & Behavioral Alignment (Syntax, Tone, and Style):** Fine-tuning is strictly superior when the goal is teaching the model a **novel linguistic syntax, rigid schema adherence, low-latency shorthand, or specialized style** (e.g., emitting a custom internal DSL, adhering to strict medical diagnosis formatting, writing unit tests matching a company repository's conventions).
- **The Hybrid Production Standard (RAG + Fine-Tuning):** Production platforms combine both: a **Fine-Tuned Small Language Model (SLM)** trained to output the company's exact JSON API schema and reasoning dialect is fed dynamic runtime passages retrieved via **RAG**.

2. Comprehensive Cost/Benefit Decision Matrix

Dimension	Prompt Engineering (In-Context)	Retrieval-Augmented Generation (RAG)	Fine-Tuning (SFT / LoRA)
Weight Modification	None ($\Delta\theta = 0$)	None ($\Delta\theta = 0$)	Yes ($\theta_{base} + \Delta\theta$)
Upfront Training Cost	**$0** (Only human design time)	Low-to-Moderate (Embedding & Vector DB setup)	Moderate-to-High ($10 – $5,000+ depending on model size & GPUs)
Per-Query Inference Cost	High (Long few-shot exemplars inflate prompt tokens)	Moderate (Retrieval adds 500–2,000 context tokens)	**Lowest** (Short zero-shot prompts; instructions baked into weights)
Inference Latency	High (Large prompt prefill phase)	Moderate (Vector DB hop ~15ms + prompt prefill)	**Fastest** (Minimal prompt prefill token count)
Dynamic Fact Adaptability	Immediate (Update the prompt string)	**Real-Time** (Update vector database index)	Extremely Slow (Requires retraining and redeploying checkpoints)
Auditability & Provenance	Low	**High** (Deterministic document citations)	Near-Zero (Black-box parametric weights)
Steerability on Style/DSL	Moderate (Instruction drift occurs)	Low (Retrieved context can confuse output styling)	**Maximum** (Model internalizes formatting rules)

3. Quantitative Decision Rubric

1. **Rule 1 (Default to Prompting):** Always prototype with zero-shot and few-shot prompt engineering first to establish baseline task feasibility.
2. **Rule 2 (RAG Gate):** If the model requires access to private company knowledge, dynamic data that changes daily, or explicit paragraph citations $\longrightarrow$ **Build RAG**.
3. **Rule 3 (Fine-Tuning Gate):** If the prompt requires 20+ few-shot examples to achieve acceptable formatting consistency, if API context costs exceed $5,000/month purely due to long system prompts, or if you must run a compact 8B model locally to match a 70B model's specialized performance $\longrightarrow$ **Execute Fine-Tuning**.

10.2 Full Fine-Tuning: Mechanics, Compute Costs, and Catastrophic Forgetting

Full Fine-Tuning (FFT) updates **100% of the model's parameters** ($\theta \in \mathbb{R}^N$) during the backward pass using standard gradient descent algorithms.

```
                    FULL FINE-TUNING MEMORY FOOTPRINT
┌──────────────────────┬──────────────────────┬──────────────────────────────┐
│    MODEL WEIGHTS     │      GRADIENTS       │       OPTIMIZER STATES       │
│       (FP16)         │        (FP16)        │        (AdamW FP32)          │
├──────────────────────┼──────────────────────┼──────────────────────────────┤
│ 2 Bytes per Parameter│ 2 Bytes per Parameter│ 12 Bytes per Parameter       │
│                      │                      │ (4B FP32 master weight +     │
│                      │                      │  4B FP32 momentum + 4B variance)│
└──────────────────────┴──────────────────────┴──────────────────────────────┘
 Total Static Memory per Parameter = 2 + 2 + 12 = 16 Bytes (Excluding KV cache &
activations)
```

1. Exact Memory Calculation Breakdown

When training with the standard AdamW optimizer in mixed precision (FP16/BF16 weights and gradients with FP32 optimizer tracking):

1. **Model Parameters (θ):** 2 bytes per parameter (FP16/BF16).
2. **Gradients (g):** 2 bytes per parameter (FP16/BF16).
3. **AdamW Optimizer States (m_t, v_t, θ_{master}):**
   - First momentum vector (m_t): 4 bytes per parameter (FP32).
   - Second variance vector (v_t): 4 bytes per parameter (FP32).
   - Master weight copy (θ_{master} for stable updates): 4 bytes per parameter (FP32).
   - Total Optimizer Memory: $4 + 4 + 4 = 12$ bytes per parameter.
4. **Static Memory Sum:**

> Static VRAM = 2 + 2 + 12 = **16 bytes per parameter**

Worked Scale Calculation (LLaMA-3 70B Full Fine-Tuning)

To train a 70-billion-parameter model:

> Static Memory = $70 \times 10^9 \times 16$ bytes ≈ 1,120 GB of VRAM

Adding **Activation Memory** ($O(B \cdot L \cdot d_{model} \cdot \text{layers})$) and workspace scratchpads requires an overhead factor of ≈ $1.25\times$:

> Total Training Memory ≈ 1,120 × 1.25 ≈ **1,400 GB of VRAM**

This workload requires a cluster of at least **18 to 24 NVIDIA H100 (80GB) GPUs** running Fully Sharded Data Parallelism (FSDP) or DeepSpeed ZeRO-3 just to perform full fine-tuning.

2. Catastrophic Forgetting

Catastrophic Forgetting (French, 1999) is the primary theoretical risk in full fine-tuning. Because all attention and MLP weights are free to update via backpropagation:

$$\theta^{(t+1)} = \theta^{(t)} - \eta \nabla_{\theta} L_{\text{task}}$$

If an instruction dataset contains only 5,000 examples of financial report drafting, the optimization landscape prioritizes minimizing loss on financial tokens. This gradient shift overwrites the pre-trained weights responsible for general multi-step logic, code generation, translation, and common-sense reasoning. Within 3 epochs, the model's performance on standard benchmarks (MMLU, GSM8K) drops significantly.

Production Mitigations for Catastrophic Forgetting

- **Experience Replay / General Data Blending:** Mix 10%–20% of high-quality, general instruction-following data (e.g., UltraChat, OpenOrca, or base pre-training tokens) into the domain dataset. This forces gradients to maintain general reasoning capabilities.
- **Weight Regularization (Weight Decay / Elastic Weight Consolidation):** Add an L_2 penalty anchoring updated weights to the base weights:

  $$L_{\text{total}} = L_{\text{task}} + \frac{\lambda}{2} \|\theta - \theta_{\text{base}}\|_2^2$$

- **Switch to Parameter-Efficient Fine-Tuning (PEFT):** Freeze the base weights completely and train low-rank adapter bypasses, mathematically eliminating parameter degradation in θ_{base}.

10.3 Dataset Preparation: Instruction Formats, Chat Templates, and Standards

Training data quality determines model behavior. Supervised Fine-Tuning (SFT) uses two primary data packaging standards: **Alpaca Format** (single-turn task execution) and **ShareGPT Format** (multi-turn conversation trees).

```
┌──────────────────────────────────┬──────────────────────────────────┐
│          ALPACA FORMAT           │          SHAREGPT FORMAT         │
│     (Single-Turn Isolation)      │    (Multi-Turn Dynamic Chat)     │
├──────────────────────────────────┼──────────────────────────────────┤
│ {                                │ {                                │
│   "instruction": "Summarize...", │   "conversations": [             │
│   "input": "Long text block...", │     {"from": "system", ...},     │
│   "output": "Summary here."      │     {"from": "human", ...},      │
│ }                                │     {"from": "gpt", ...},        │
│                                  │     {"from": "human", ...},      │
│ Optimal for: Extraction, simple  │     {"from": "gpt", ...}         │
│ deterministic transformations.   │   ]                              │
│                                  │ }                                │
│                                  │ Optimal for: Chatbots, agents.   │
└──────────────────────────────────┴──────────────────────────────────┘
```

1. Alpaca Format (Single-Turn)

Originated by Stanford Alpaca, this format enforces a rigid structural schema:

- `instruction`: String defining the general task to be performed.
- `input`: (Optional) String providing specific context, source text, or variables for the instruction.
- `output`: Ground-truth target completion string.

```
{
  "instruction": "Classify the sentiment of the following financial disclosure as BULLISH,
BEARISH, or NEUTRAL.",
  "input": "Q3 operating revenue expanded 28% year-over-year to $4.2B, surpassing analyst
expectations.",
  "output": "BULLISH"
}
```

2. ShareGPT Format (Multi-Turn Conversations)

Standardized by open-source communities (FastChat, Axolotl, LLaMA-Factory) to train conversational agents across multiple dialogue turns:

- Uses a `conversations` array containing ordered message maps with keys `from` (`system`, `human`, `gpt`, or `tool`) and `value`.

```
{
  "conversations": [
    {
      "from": "system",
      "value": "You are a Linux systems engineering tutor. Provide concise, POSIX-compliant
terminal commands."
    },
    {
      "from": "human",
      "value": "How do I locate and display all listening TCP sockets on port 8080?"
    },
    {
      "from": "gpt",
      "value": "Execute: `ss -tulpn | grep :8080` or `lsof -iTCP:8080 -sTCP:LISTEN`."
    },
    {
      "from": "human",
      "value": "What flag restricts lsof from resolving hostnames to avoid DNS latency?"
    },
    {
      "from": "gpt",
      "value": "Add the `-n` flag: `lsof -nP -iTCP:8080 -sTCP:LISTEN`."
    }
  ]
}
```

3. Chat Templates & Jinja2 Serialization

An LLM does not ingest JSON dictionaries directly; it accepts a 1D sequence of token IDs. Modern tokenizers use **Jinja2 Chat Templates** to compile structured message dictionaries into a flat text stream punctuated by model-specific control tokens.

Example: ChatML vs. LLaMA-3 Template Serialization

```
{# Jinja2 Chat Template Representation #}
{% for message in messages %}
  {% if message['role'] == 'system' %}
    {{ '<|start_header_id|>system<|end_header_id|>\n\n' + message['content'] + '<|eot_id|>'
}}
  {% elif message['role'] == 'user' %}
    {{ '<|start_header_id|>user<|end_header_id|>\n\n' + message['content'] + '<|eot_id|>' }}
  {% elif message['role'] == 'assistant' %}
    {{ '<|start_header_id|>assistant<|end_header_id|>\n\n' + message['content'] +
'<|eot_id|>' }}
  {% endif %}
{% endfor %}
```

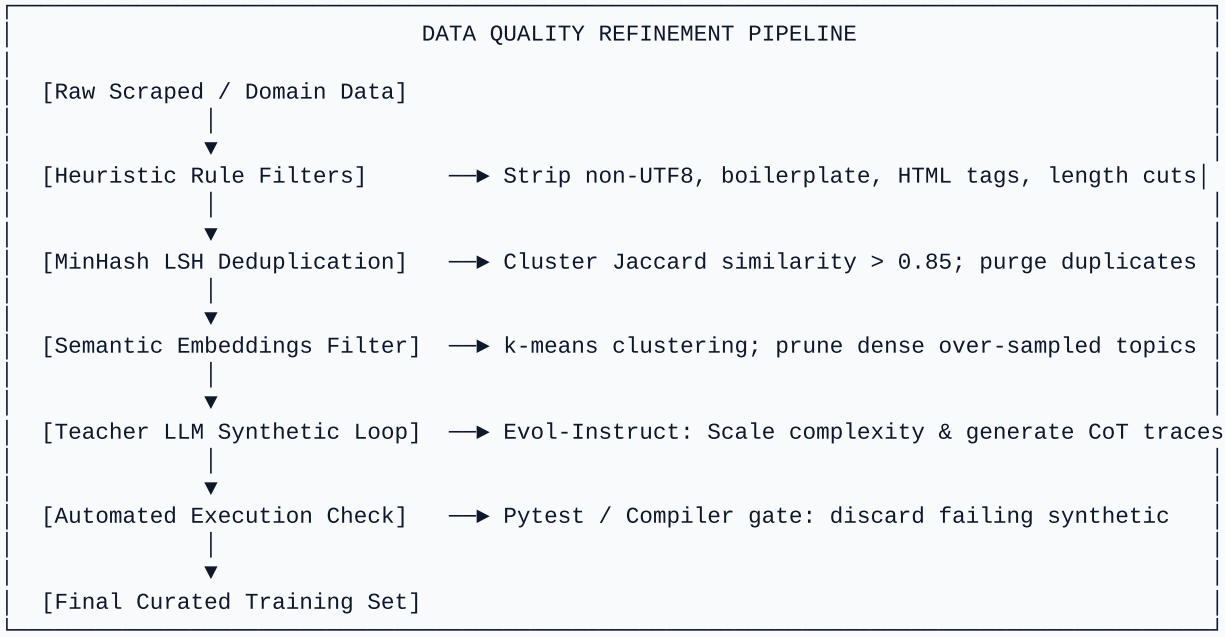
4. Loss Masking on Input Tokens (Data Packing Rule)

When fine-tuning on multi-turn conversations, the loss must **only be calculated on the assistant's tokens**.

- If loss is computed on the user's prompt tokens, the model wastes gradient updates memorizing the statistical distribution of user questions rather than learning how to respond.
- **Implementation:** Set the PyTorch loss label for all system and user prompt tokens to `-100` (`CrossEntropyLoss(ignore_index=-100)`). The loss function zeroes out gradients for all token positions labeled `-100`, backpropagating strictly on the tokens emitted by the `assistant`.

10.4 Data Quality, Deduplication, and Synthetic Generation via Teacher LLMs

In post-Chinchilla scaling, **data quality outweighs raw dataset volume**. Training on 5,000 meticulously verified, diverse instruction examples produces better downstream reasoning than training on 500,000 noisy, scraped examples.



1. MinHash LSH (Locality-Sensitive Hashing) Deduplication

Near-duplicate instruction examples waste training compute and cause the model to repeat boilerplate phrases. **MinHash LSH** identifies near-duplicate texts in $O(N)$ time:

1. ***k*-Shingling:** Convert each document into a set of contiguous character or word shingles (e.g., $k = 5$).
2. **MinHashing:** Compute M independent hash functions (e.g., $M = 128$) across the shingle set. The minimum hash value for each function forms a compact MinHash signature vector.
3. **Locality-Sensitive Hashing:** Partition the signature vectors into b bands of r rows. Documents that collide in any single band bucket are flagged as candidate duplicates.
4. **Jaccard Similarity Pruning:** If candidate pairs have an estimated Jaccard similarity $J(A, B) = |A \cap B| / |A \cup B| \geq 0.85$, drop one of the instances.

2. Synthetic Data Generation via Teacher Models (Evol-Instruct)

Pioneered by WizardLM (Xu et al., 2023), **Evol-Instruct** uses a frontier "Teacher" model (e.g., GPT-4o, Claude 3.5 Sonnet) to iteratively upgrade simple instructions into complex, multi-hop reasoning tasks.

The Five Evolutionary Operators

1. **Add Constraints:** Introduce strict operational boundaries (*"Rewrite this Python script, but ensure time complexity is O(N \log N) and no third-party libraries are imported"*).
2. **Deepen Domain Knowledge:** Require advanced domain-specific mechanics (*"Explain how a Redis cluster handles network partitions using Raft-based consensus instead of general terms"*).
3. **Concretize & Ground:** Replace abstract descriptions with concrete, real-world edge cases.
4. **Increase Reasoning Steps:** Convert single-step lookup queries into multi-step deductive chains.
5. **Mutation (Generate Hard Counter-Cases):** Instruct the teacher model to write an adversarial edge case that breaks naive implementations.

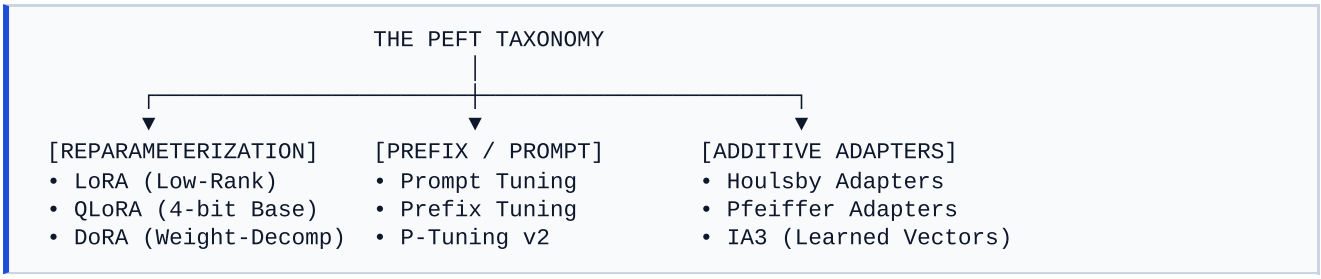
Execution-Gated Synthetic Verification

Never trust teacher LLM generations blindly. Synthetic programming or mathematical datasets must pass an **automated verification gate**:

- Run generated Python scripts inside an isolated WASM/Docker sandbox against synthetic unit tests (`assert solution(...) == expected`).
- If the code throws an exception, discard the synthetic example automatically. This ensures only mathematically and syntactically verified reasoning traces enter the SFT training set.

10.5 The Parameter-Efficient Fine-Tuning (PEFT) Landscape

Parameter-Efficient Fine-Tuning freezes the overwhelming majority of base model parameters (θ_{base}) and optimizes only a small set of auxiliary parameters ($\Delta\theta$), typically representing ***0.01% – 2.0% of the base parameter count***.



1. Additive Adapters (Houlsby & Pfeiffer)

- **Mechanics:** Insert small, bottleneck feed-forward sub-networks directly into each Transformer layer (either after multi-head attention, after the FFN block, or both).
- **Architecture:** The bottleneck projects the hidden dimension down to a smaller dimension $m \ll d_{model}$ via down-projection W_{down}, applies a non-linear activation σ, and projects back up via W_{up}:

$$\text{Adapter}(h) = \sigma(h\, W_{\text{down}})\, W_{\text{up}} + h$$

- **Primary Drawback:** Adds sequential execution steps into every Transformer block, introducing noticeable **inference latency overhead (15%–30%)** that cannot be mathematically merged into the base weights.

2. Prefix Tuning (Li & Liang, 2021)

- **Mechanics:** Prepends continuous, learnable virtual Key-Value vectors directly to the Transformer's attention layers at every layer:

$$\text{Attention}(Q, [P_K\,;\,K], [P_V\,;\,V])$$

  where $P_K, P_V \in \mathbb{R}^{l \times d_k}$ are trainable prefix parameters of virtual token length l, while base keys K and values V remain frozen.
- **Drawback:** Reduces the available sequence length for user tokens in the context window and is prone to optimization instabilities during training.

3. Prompt Tuning (Lester et al., 2021)

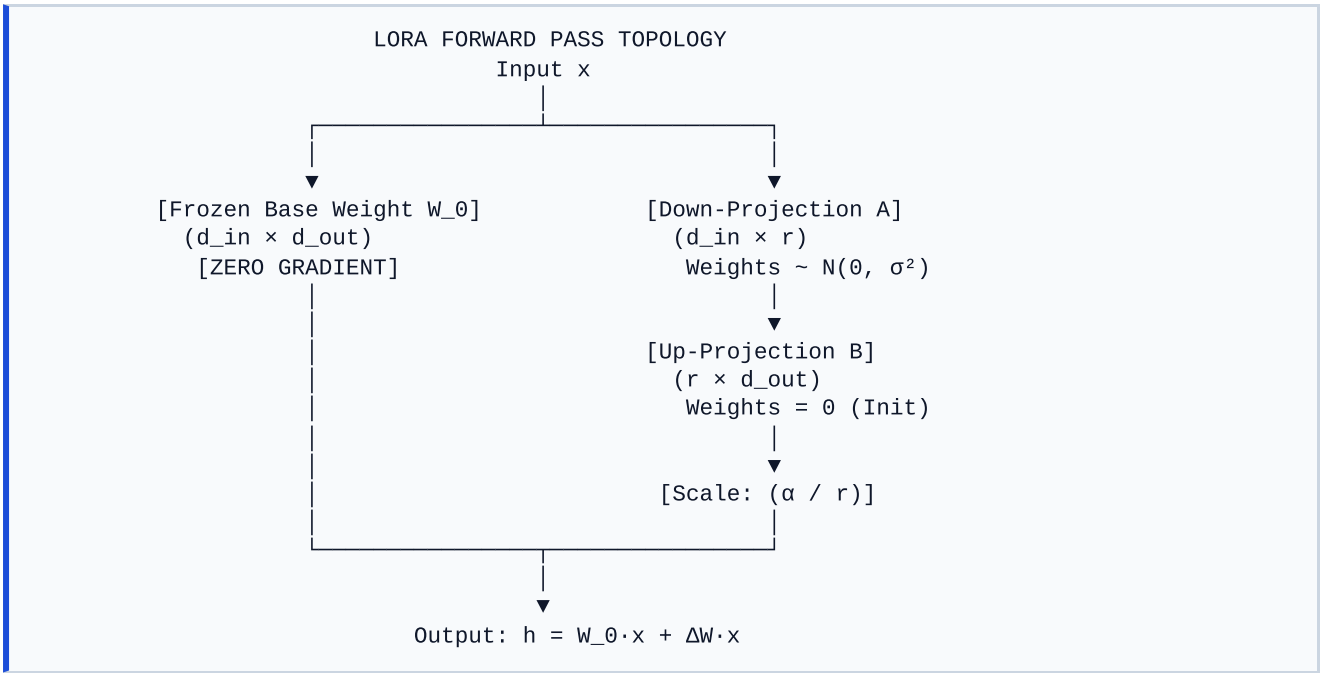
- **Mechanics:** Simplifies Prefix Tuning by prepending k learnable continuous embedding vectors **only at the input embedding layer** ($X' = [P\ ;\ X]$). The Transformer layers remain completely frozen.
- **Limitation:** Fails to steer models below 10B parameters effectively. While parameter-efficient, it lacks the expressive capacity required for complex reasoning adjustments.

4. P-Tuning v2 (Liu et al., 2021)

- **Mechanics:** Deep Prompt Tuning that introduces learnable prefix tokens across all Transformer layers independently (acting similarly to Prefix Tuning, but without reparameterization MLPs), optimized specifically for natural language understanding and sequence labeling.

10.6 Low-Rank Adaptation (LoRA): Mathematical Formulation and Architecture

Introduced by Hu et al. (Microsoft, 2021), **Low-Rank Adaptation (LoRA)** is the standard PEFT method across industry. It achieves full fine-tuning performance without adding inference latency or altering the base architecture.



1. The Low-Rank Hypothesis and Mathematical Derivation

The foundational hypothesis of LoRA posits that **the weight updates ΔW during task adaptation possess a low "intrinsic dimension"**—meaning the meaningful rank of the gradient update matrix is significantly smaller than the full rank of the weight matrix itself.

Given a frozen weight matrix $W_0 \in \mathbb{R}^{d_{out} \times d_{in}}$, LoRA constrains its update ΔW by factorizing it into two low-rank matrices:

$$\Delta W = B \cdot A$$

where:

- $A \in \mathbb{R}^{r \times d_{in}}$ (Down-projection matrix)
- $B \in \mathbb{R}^{d_{out} \times r}$ (Up-projection matrix)
- $r \ll \min(d_{in}, d_{out})$ is the **intrinsic rank** hyperparameter (typically $r \in [8, 64]$).

The Modified Forward Pass

For an incoming hidden state vector $x \in \mathbb{R}^{d_{in}}$, the modified forward pass is:

$$h = W_0 x + \Delta W x = W_0 x + \frac{\alpha}{r} (B A) x$$

2. Initialization Strategy (Zero Identity Shift)

To guarantee that training begins from the base model's exact initial state without random perturbations:

1. Matrix A is initialized using a **Gaussian random distribution**: $A \sim \mathcal{N}(0, 1 / r)$.
2. Matrix B is initialized to **exact zeros**: $B = 0$.
3. Therefore, at step zero:

   $$\Delta W = B \cdot A = 0 \cdot A = \mathbf{0}$$

   The adapter outputs zero change, ensuring the initial forward pass is identical to the base pre-trained model.

3. The Scaling Factor α (Alpha) and Learning Dynamics

The scalar multiplier α / r stabilizes optimization:

- α (Alpha) is a constant hyperparameter.
- When experimenting with different ranks (e.g., increasing r from 8 to 64), the scale factor α / r prevents the magnitude of the adapter activations from exploding or collapsing, eliminating the need to retune the learning rate when scaling rank.
- **Standard Production Heuristic:** Set $\alpha = 2 \times r$ or $\alpha = r$.

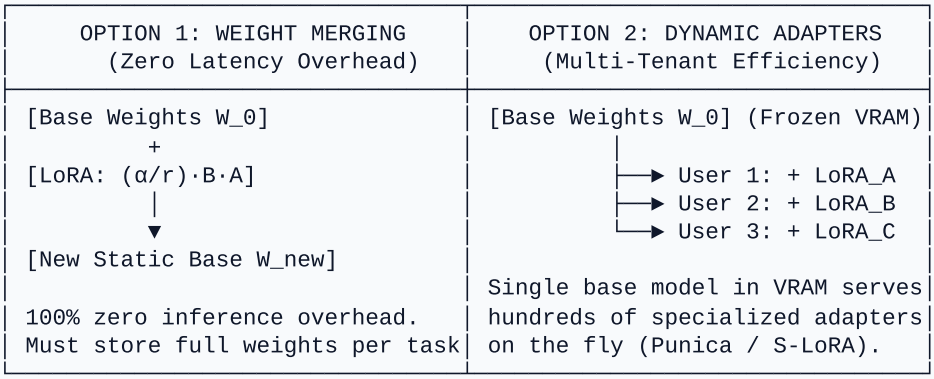
4. Target Module Selection: Where Should LoRA Be Applied?

In early implementations, LoRA was applied exclusively to the attention projections (W_Q, W_V). Research (e.g., QLoRA paper, Dettmers et al.) shows that **applying LoRA across all linear layers yields the highest performance, closely matching full fine-tuning**:

- **Attention Blocks:** W_Q, W_K, W_V, W_O (Encodes routing and relational attention patterns).
- **MLP / Feed-Forward Blocks:** W_{gate}, W_{up}, W_{down} (SwiGLU layers — stores domain knowledge and factual associations).
- **Rule:** An adapter trained with rank $r = 16$ across all linear layers outperforms an adapter trained with rank $r = 64$ placed only on W_Q, W_V, while using fewer parameters.

10.7 LoRA Merging vs. Multi-Adapter Serving

Deploying fine-tuned models in production involves choosing between **weight merging** (zero-latency single-tenant serving) and **dynamic adapter swapping** (low-memory multi-tenant serving).



1. LoRA Weight Merging (Zero Latency Overhead)

Because matrix multiplication is distributive:

$$h = W_0 x + \left(\frac{\alpha}{r} B A\right) x = \left(W_0 + \frac{\alpha}{r} B A\right) x = W_{\text{merged}} \cdot x$$

You can compute the inner product $(\alpha / r) B A$ and add it directly to the base weights W_O in FP16/BF16 precision:

$$W_{\text{merged}} = W_0 + \frac{\alpha}{r} (B \cdot A)$$

- **Advantage:** The adapter matrices A and B are discarded. The runtime model is a standard Transformer checkpoint. There is **zero inference latency penalty, zero memory overhead, and no specialized runtime required**.
- **Limitation:** Permanently mutates weights. If you have 20 task-specific models, you must store 20 full 70B checkpoints on disk and host them on separate GPU instances.

```
from peft import PeftModel
from transformers import AutoModelForCausalLM

# Load base model in FP16/BF16
base_model = AutoModelForCausalLM.from_pretrained("meta-llama/Meta-Llama-3-8B-Instruct")

# Mount trained LoRA adapter
model_with_adapter = PeftModel.from_pretrained(base_model, "./lora_adapter_checkpoint")

# Permanently fuse weights into base parameters
merged_model = model_with_adapter.merge_and_unload()

# Save as a standard, standalone HuggingFace checkpoint
merged_model.save_pretrained("./llama3_merged_production")
```

2. Multi-Adapter Dynamic Serving (S-LoRA / Punica Architecture)

In multi-tenant SaaS environments, keeping 50 distinct fine-tuned 70B models running concurrently is economically infeasible. Modern serving engines (such as **vLLM**, **S-LoRA**, and **Punica**) dynamically route requests to different adapters using a single shared base model:

1. **Unified Base Model in VRAM:** The base weights W_0 are loaded into GPU memory once (e.g., consuming 140 GB for a 70B model).
2. **Adapters in Host RAM:** Hundreds of small LoRA adapters (A and B take only 20–100 MB each) reside in CPU system RAM.
3. **Dynamic Paged Fetching:** When Request A (requiring `legal_adapter`) and Request B (requiring `code_adapter`) arrive in the same batch:
   - The base model forward pass computes $W_0 x$ in parallel.
   - Specialized CUDA kernels compute the respective low-rank paths: $(\alpha / r)\,(B_{legal} A_{legal})\, x_A$ and $(\alpha / r)\,(B_{code} A_{code})\, x_B$.
   - Results are summed before the next Transformer block.

- **Economic Advantage:** A single GPU cluster serves hundreds of specialized, fine-tuned customer models at the cost of running a single base model.

Technical Summary of Module 10 Part 1

Section	Core Mechanism	Critical Formula / Invariant	Primary Failure Mode Addressed
10.1 Adaptation Matrix	Tradeoff: Knowledge (RAG) vs. Form/Style (Fine-Tuning)	SFT cost $\propto \theta$; Prompt cost $\propto \text{tokens}^2$	Misapplying SFT for memorizing volatile enterprise facts
10.2 Full Fine-Tuning	All weights update; 16 bytes/param static VRAM in AdamW	$M_{static} = 16 \times N_{params}$ bytes	Catastrophic forgetting; cluster VRAM exhaustion

Section	Core Mechanism	Critical Formula / Invariant	Primary Failure Mode Addressed
10.3 Dataset Formats	Alpaca (single-turn) vs ShareGPT (multi-turn) Jinja templates	`CrossEntropyLoss(ignore_index=-100)` on prompts	Loss degradation from model memorizing user prompt tokens
10.4 Data Quality	MinHash LSH deduplication; Evol-Instruct synthetic scaling	Jaccard threshold $J(A, B) \geq 0.85$	Model repeating noise or boilerplate from duplicate data
10.5 PEFT Landscape	Freezes θ_{base}; updates small parameter subset $\Delta\theta$	Adapters vs. Prefix vs. Reparameterization	Inference latency penalties introduced by sequential adapters
10.6 LoRA Mechanics	Factorizes weight updates into low-rank matrices: $\Delta W = B \cdot A$	$h = W_0 x + (\alpha / r)(B A) x$; $B=0, A \sim \mathcal{N}$	VRAM explosion during fine-tuning runs
10.7 Weight Merging	$W_{merged} = W_0 + (\alpha / r)(B A)$ vs. Dynamic multi-adapter serving	Distributive property of matrix multiplication	Infrastructure bloat from hosting multiple full checkpoints